CMPU2032 - Programming & Algorithms 2 (2025/26)

Extra Credit Report

Student Name: *Dwight Egerton*

Student ID: *D24127624*

Release Date: *16 April 2026*

Submission Date: *3 May 2026 @ 23:59*

Overview

This is an extra credit assignment; although it is graded out of 10 marks, the overall course total remains capped at 100, so the maximum achievable overall mark will still be 100.

Table of Contents

My Submission Summary

I reused the MENU structure from my PBA submission. This could have been managed in a separate library so the code would not be duplicated, but to keep the submission simple, I just copied the files.

I created a seperation of code logically to keep code more managable and extendable.

- `repository/`: code related to accessing the backend repository database
- `service/`: manage application mapping to repository CRUD operations
- `menu/`: the comand-line interface

When the application initialises, the backend repositiory, service, and menu are initialised. Also, at this time, when the backend repository is initialised, any required tables will be created/initialised.

Repository

In my solution I have created a `EmployeeDatabse` interface that can be implemented as needed. This interface provides all the basic CRUD operations needed to manage an `employee` record.

As required, a SQLite database is to be used to store and manage the `employee` tabe and data. This is by the `SqliteEmployeeDatabase` class that implements the `EmployeeDatabse` interface.

In the `initialise_database()` method an backend tables, indexes, etc. can be initialised/crreated. To ensure this can be done everytime the program launches, I use SQL logic only update `IF NOT EXISTS`, for example: -

```
CREATE TABLE IF NOT EXISTS employees (
    employee_id INTEGER         PRIMARY KEY AUTOINCREMENT,
    name        VARCHAR(255)    NOT NULL,
    dob         DATE            NOT NULL,
    department  VARCHAR(255),
    salary      FLOAT           NOT NULL
)
```

Service

The service layer provides a place to implement any business logic required before accessing the backend repository. As this only implements a very simple set of use-cases, there isn't any additional business-logic to add in the service layer. So this is just a mapping of the service to the respective repository CRUD operations.

The `EmployeeService` will use an implementation of the `EmployeeDatabse` interface to connect to the backend repository.

Menu

As mentioned, I resued the menu logic from my PBA submission, to create a simple command-line interface for this project. Each menu-option is implements the `MenuOption` interface and the menu is constructed through the `MenuBuilder`, that implements the ***Builder*** design-pattern.

This provides the following menu options: -

1. Add new employee record
2. Display all employee records
3. Update an employee record
4. Remove an employee record

Setup and Execution

Create Virtual Environment

To ensure this project does not impact any other parts of your system, a virtual environment should be setup. The assumption is that you have Python (version 3.11 or later) installed on your computer. This can simple be done by executing the following commands: -

- Windows ...

```
python -m venv .venv
.venv\Scripts\activate
```

- Mac, Linux or WSL ...

```
python3 -m venv .venv
source .venv/bin/activate
```

This can be done in other ways, for example: through your IDE or with tools like `pipx`. Please refer to the respective documentation for any other approach you wish to follow.

Install dependencies

This project uses `Poetry` for dependency management, building, running and testing. Run the commands below to install `Poetry` into your Python virtual-environment as-well-as all the required project dependencies: -

```
pip3 install poetry
poetry install
```

Launching the `Employee Management System` Application

To run the command-line interface: -

```
poerty run main
```

Executing all Application Code Unit-Tests

To run the unit tests: -

```
poetry run tests
```